

Arquitectura de Datos

Tema 3: Redis

Bases de datos clave-valor en memoria

Curso académico 2026/2027



Parte A Contador distribuido



Objetivo

Implementar **contadores distribuidos atómicos** con Redis para generar **IDs únicos globales** en un entorno multi-instancia. Aprenderás a usar INCR, INCRBY y DECR para resolver el problema de generación de identificadores en sistemas distribuidos, evitando colisiones sin necesidad de bloqueos pesados.

Al finalizar esta parte serás capaz de:

- Generar IDs secuenciales únicos globales con INCR.
- Gestionar contadores independientes por tipo de entidad.
- Comprender la atomicidad de las operaciones de incremento en concurrencia.
- Implementar un generador de IDs distribuido en C# con StackExchange.Redis.
- Implementar un generador de IDs distribuido en Python con redis-py.
- Diseñar una estrategia de resiliencia con fallback a MariaDB.

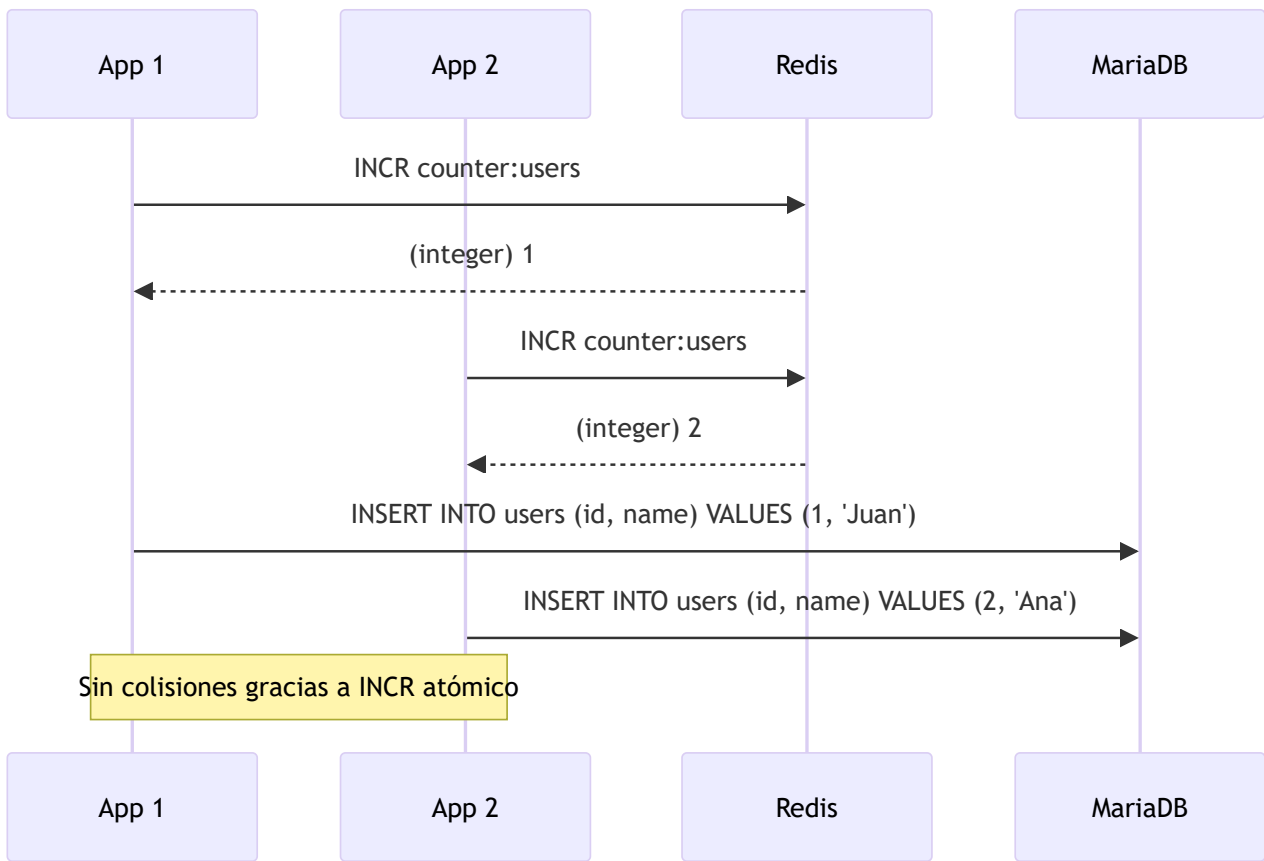
Requisitos previos

Componente	Versión mínima	Comprobación
Docker Desktop	4.x	<code>docker --version</code>
JetBrains DataGrip	2024.x	Abrir la aplicación
.NET SDK	10.x	<code>dotnet --version</code>
Python	3.12+	<code>python --version</code>
Redis (imagen Docker)	7.x	<code>docker pull redis:7-alpine</code>

Nota: Debes haber completado los laboratorios **PR1** (Introducción a Redis + Docker) y **PR2** (Estructuras de datos) antes de empezar esta parte. Asegúrate de que el contenedor Redis está en ejecución.

Arquitectura del laboratorio

Cuando múltiples instancias de aplicación necesitan generar IDs únicos, Redis actúa como **generador centralizado de secuencias**. El comando INCR es atómico, lo que garantiza que dos clientes concurrentes **nunca** obtienen el mismo valor.



Explicación: Redis ejecuta todos los comandos en un **único hilo**. Cuando ejecuta INCR, nadie más puede ejecutar comandos hasta que termine. Esto garantiza atomicidad sin necesidad de bloqueos distribuidos como `SELECT ... FOR UPDATE` en bases de datos relacionales.

Paso 1: IDs únicos con INCR

El comando INCR incrementa atómicamente el valor de una clave numérica. Si la clave no existe, la crea con valor **1**. Esta propiedad lo convierte en el mecanismo ideal para generar IDs secuenciales únicos.

Generar 10 IDs secuenciales para usuarios

```
# Contador distribuido para generar IDs únicos
# INCR es atómico: garantiza que dos clientes concurrentes nunca obtienen el mismo valor

# Generar 10 IDs secuenciales para usuarios
INCR counter:users
# Output: (integer) 1

INCR counter:users
# Output: (integer) 2

INCR counter:users
# Output: (integer) 3

INCR counter:users
# Output: (integer) 4

INCR counter:users
# Output: (integer) 5

INCR counter:users
# Output: (integer) 6

INCR counter:users
# Output: (integer) 7

INCR counter:users
# Output: (integer) 8

INCR counter:users
# Output: (integer) 9

INCR counter:users
# Output: (integer) 10

# Verificar el último ID generado
GET counter:users
# Output: "10"
```

Salida esperada:

Resultado

```
(integer) 1
(integer) 2
(integer) 3
(integer) 4
(integer) 5
(integer) 6
(integer) 7
(integer) 8
(integer) 9
(integer) 10
"10"
```

Consejo: No necesitas inicializar la clave con SET. Si la clave no existe, INCR la crea automáticamente con valor 1. Esto simplifica la lógica de la aplicación.

Paso 2: Contadores separados por entidad

En una aplicación real, cada tipo de entidad (usuarios, pedidos, productos) necesita su propio contador. Redis permite gestionar múltiples contadores de forma independiente utilizando **claves diferentes** para cada entidad.

Contadores separados por tipo de entidad

```
# Contadores separados por tipo de entidad
# Cada clave mantiene su propio contador de forma independiente

INCR counter:users
# Output: (integer) 1

INCR counter:orders
# Output: (integer) 1

INCR counter:products
# Output: (integer) 1

# Cada contador es independiente: incrementar uno no afecta a los demás
INCR counter:users
# Output: (integer) 2

# INCRBY para incrementar en valores personalizados
# Útil cuando necesitas reservar un bloque de IDs
INCRBY counter:users 5
# Output: (integer) 6

# DECR para decrementar
# Puede usarse para corregir errores o liberar IDs reservados
DECR counter:users
# Output: (integer) 5

# Verificar estado actual de cada contador
GET counter:users
# Output: "5"

GET counter:orders
# Output: "1"

GET counter:products
# Output: "1"
```

Salida esperada:

Resultado

```
(integer) 1
(integer) 1
(integer) 1
(integer) 2
(integer) 6
(integer) 5
"5"
"1"
"1"
```

Advertencia: DECR puede producir valores negativos si se usa sin control. En producción, valida siempre que el contador no baje de un umbral mínimo antes de asignar el ID.

Paso 3: Simular concurrencia

Para demostrar que INCR es atómico, simula dos clientes generando IDs simultáneamente. Abre **dos terminales** conectadas a Redis y ejecuta los comandos de forma alterna.

3.1 Cliente A — Terminal 1 (DataGrip)

Terminal 1 – Cliente A

```
# Terminal 1: Primer cliente
# Conectarse a Redis
redis-cli

# Resetear el contador para la prueba
SET counter:demo 0

# Simular operación del cliente A
INCR counter:demo
# Output: (integer) 1

# ... esperar un momento (otro cliente incrementa) ...

INCR counter:demo
# Output: (integer) 3  (no 2, porque el cliente B ya incrementó)
```

3.2 Cliente B — Terminal 2 (Redis CLI)

Terminal 2 – Cliente B

```
# Terminal 2: Segundo cliente (concurrente)
# Conectarse a Redis (otra sesión)
redis-cli

# Simular operación del cliente B (mientras A espera)
INCR counter:demo
# Output: (integer) 2

INCR counter:demo
# Output: (integer) 4
```

Salida esperada:

Resultado

```
# Terminal 1 (Cliente A):
```

```
(integer) 1
```

```
(integer) 3
```

```
# Terminal 2 (Cliente B):
```

```
(integer) 2
```

```
(integer) 4
```

Resultado esperado: Los IDs generados son **1, 2, 3, 4** — sin repeticiones ni colisiones. Gracias a la atomicidad de INCR, cada cliente recibe un valor único y secuencial, incluso ejecutando comandos de forma concurrente.

Paso 4: Implementación en C# con StackExchange.Redis

StackExchange.Redis es el cliente .NET más utilizado para Redis. Proporciona una API asíncrona completa con soporte para pub/sub, transacciones y operaciones atómicas como `StringIncrement`.

```
// .NET 10 + C# 14: Contador distribuido con Redis
// Generador de IDs únicos globales usando INCR atómico

using StackExchange.Redis;

// Conexión al servidor Redis
// ConnectionMultiplexer gestiona el pool de conexiones de forma eficiente
using var redis = await ConnectionMultiplexer.ConnectAsync("localhost:6379");
var db = redis.GetDatabase();

// Generador de IDs distribuido
// INCR es atómico: garantiza unicidad incluso en concurrencia
long GenerateId(string entity)
{
    // StringIncrement: incrementa atómicamente el contador en 1
    // Si la clave no existe, la crea con valor 1
    return db.StringIncrement($"counter:{entity}");
}

// Uso: generar IDs para diferentes entidades
var userId = GenerateId("users");
var orderId = GenerateId("orders");
var productId = GenerateId("products");

Console.WriteLine($"ID usuario: {userId}"); // Output: ID usuario: 1
Console.WriteLine($"ID pedido: {orderId}"); // Output: ID pedido: 1
Console.WriteLine($"ID producto: {productId}"); // Output: ID producto: 1

// Generar más IDs para usuarios
for (int i = 0; i < 5; i++)
{
    var id = GenerateId("users");
    Console.WriteLine($"ID usuario: {id}"); // Output: 2, 3, 4, 5, 6
}

// Ejemplo: crear usuario con ID generado por Redis
var nuevoId = GenerateId("users");
Console.WriteLine($"INSERT INTO users (id, name) VALUES ({nuevoId}, 'Ana')");

// Consultar el estado actual de un contador
var contadorActual = db.StringGet("counter:users");
Console.WriteLine($"Total usuarios generados: {contadorActual}");

// Incrementar en una cantidad personalizada (reservar bloque de IDs)
var bloque = db.StringIncrement("counter:users", 10);
Console.WriteLine($"Próximo bloque empieza en: {bloque}");
```

Salida esperada:

Resultado

```
ID usuario: 1
ID pedido: 1
ID producto: 1
ID usuario: 2
ID usuario: 3
ID usuario: 4
ID usuario: 5
ID usuario: 6
INSERT INTO users (id, name) VALUES (7, 'Ana')
Total usuarios generados: 7
Próximo bloque empieza en: 17
```

Explicación: StringIncrement es la versión .NET de INCR. Es atómica y O(1). El método StringIncrement con un segundo parámetro permite incrementar en una cantidad personalizada, ideal para reservar bloques de IDs.

Paso 5: Alternativa en Python con redis-py

redis-py es el cliente oficial de Python para Redis. Ofrece un API intuitiva con soporte completo para todas las estructuras de datos y operaciones atómicas. Esta es una alternativa si prefieres Python sobre C#.

Generador de IDs distribuido en Python

```
# redis-py: Contador distribuido con Redis
# Generador de IDs únicos globales usando INCR atómico

import redis

# Conexión a Redis
# decode_responses=True convierte bytes a strings automáticamente
r = redis.Redis(host='localhost', port=6379, decode_responses=True)

def generate_id(entity: str) -> int:
    """Generar ID único para una entidad usando INCR atómico."""
    # incr: incrementa atómicamente el contador
    # Si la clave no existe, la crea con valor 1
    return r.incr(f"counter:{entity}")

# Uso: generar IDs para diferentes entidades
user_id = generate_id("users")
order_id = generate_id("orders")
product_id = generate_id("products")

print(f"ID usuario: {user_id}")      # Output: ID usuario: 1
print(f"ID pedido: {order_id}")      # Output: ID pedido: 1
print(f"ID producto: {product_id}")  # Output: ID producto: 1

# Generar más IDs para usuarios
for i in range(5):
    user_id = generate_id("users")
    print(f"ID usuario: {user_id}")  # Output: 2, 3, 4, 5, 6

# Ejemplo: crear usuario con ID generado por Redis
nuevo_id = generate_id("users")
print(f"INSERT INTO users (id, name) VALUES ({nuevo_id}, 'Ana')")

# Consultar el estado actual de un contador
contador_actual = r.get("counter:users")
print(f"Total usuarios generados: {contador_actual}")

# Incrementar en una cantidad personalizada (reservar bloque de IDs)
bloque = r.incr("counter:users", amount=10)
print(f"Próximo bloque empieza en: {bloque}")
```

Salida esperada:

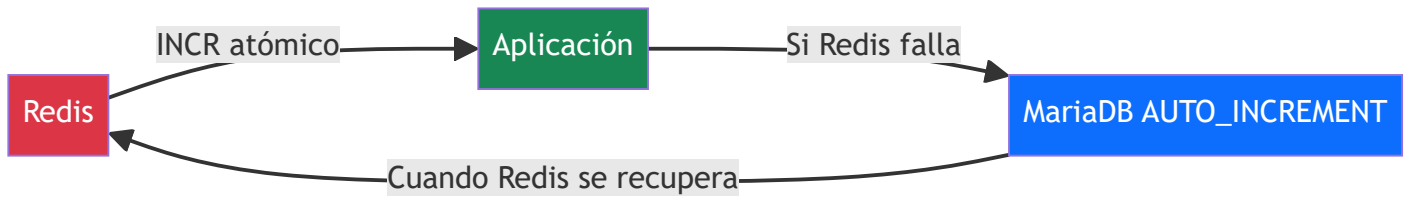
Resultado

```
ID usuario: 1
ID pedido: 1
ID producto: 1
ID usuario: 2
ID usuario: 3
ID usuario: 4
ID usuario: 5
ID usuario: 6
INSERT INTO users (id, name) VALUES (7, 'Ana')
Total usuarios generados: 7
Próximo bloque empieza en: 17
```

Consejo: En producción, encapsula `r.incr()` en una función con reintentos. Si Redis falla temporalmente, la aplicación puede reintentar sin perder atomicidad. Consulta el Paso 6 para una estrategia de resiliencia completa.

Paso 6: Estrategia de resiliencia — Fallback a MariaDB

En producción, Redis puede caer. Una estrategia de resiliencia combina Redis como fuente principal de IDs con **MariaDB como fallback**. Cuando Redis se recupera, se sincronizan los contadores para evitar futuras colisiones.



Estrategia de fallback

```
# Estrategia de resiliencia: si Redis cae, usar MariaDB como fallback
# En producción, esto se implementa con un try-catch en la aplicación

# 1. Intentar con Redis (fuente principal)
INCR counter:users
# Output: (integer) 1

# 2. Si Redis falla, usar AUTO_INCREMENT de MariaDB
# INSERT INTO users (id, name) VALUES (DEFAULT, 'Juan')
# MariaDB genera el ID automáticamente con su secuencia interna

# 3. Cuando Redis se recupere, sincronizar contadores
# Obtener el último ID de MariaDB
# SELECT MAX(id) FROM users; → 42

# Establecer el contador de Redis al valor de MariaDB
SET counter:users 42
# A partir de aquí, Redis continúa generando IDs desde 43
```

```
// .NET 10 + C# 14: Top-level statements
using StackExchange.Redis;

// Conexión Redis reutilizable
using var redis = await ConnectionMultiplexer.ConnectAsync("localhost:6379");
var database = redis.GetDatabase();

// Generador de IDs con fallback a MariaDB
long GenerateIdWithFallback(string entity, IDbConnection dbConnection)
{
    try
    {
        // Intentar con Redis (fuente principal)
        return database.StringIncrement($"counter:{entity}");
    }
    catch (RedisConnectionException)
    {
        // Fallback: usar AUTO_INCREMENT de MariaDB
        using var command = dbConnection.CreateCommand();
        command.CommandText = "SELECT IFNULL(MAX(id), 0) + 1 FROM users";
        return (long)command.ExecuteScalar();
    }
}

// Después de la recuperación de Redis, sincronizar:
void SyncCounterFromDb(string entity, IDbConnection dbConnection)
{
    using var command = dbConnection.CreateCommand();
    command.CommandText = $"SELECT IFNULL(MAX(id), 0) FROM {entity}";
    var lastId = (long)command.ExecuteScalar();

    // Establecer el contador de Redis al último ID de MariaDB
    database.StringSet($"counter:{entity}", lastId);
}
```

Advertencia: La sincronización después de un fallback debe ser **atómica**. Si múltiples instancias intentan sincronizar simultáneamente, usa una transacción Redis (MULTI/EXEC) o un bloqueo distribuido para evitar sobrescribir el contador con un valor antiguo.

Preguntas de reflexión

Antes de finalizar, reflexiona sobre las siguientes preguntas para consolidar lo aprendido:

1. ¿Por qué INCR es mejor que GET + SET + 1 para contadores?

Piensa en qué pasaría si dos clientes leen el mismo valor simultáneamente. ¿Cómo resuelve Redis la concurrencia con INCR?

2. ¿Qué pasaría si Redis cae después de incrementar el contador pero antes de que la aplicación use el ID?

¿Se perdería el ID? ¿Cómo afectaría esto a la integridad de los datos en MariaDB?

3. ¿Cómo recuperarías el contador si Redis se pierde completamente (sin persistencia)?

Considera el caso de un fallo del contenedor Docker sin volumen persistente. ¿Qué estrategia usarías para reconstruir el contador?

4. ¿Qué alternativas existen a Redis para IDs distribuidos (UUID, Snowflake, ULID)?

¿En qué casos usarías un UUID en lugar de un ID secuencial? ¿Qué ventajas e inconvenientes tiene cada enfoque?

5. ¿Cómo manejarías aislamiento con contadores separados por vendedor?

Si cada vendedor de tu tienda online necesita sus propios IDs, ¿cómo estructurarías las claves en Redis para mantener la separación?

Comandos de referencia rápida



Comando	Descripción	Ejemplo
INCR	Incrementar contador en 1 (atómico)	INCR counter:users
INCRBY	Incrementar contador en N	INCRBY counter:users 10
DECR	Decrementar contador en 1 (atómico)	DECR counter:users
SET ... NX	Crear clave solo si no existe	SET lock:resource 1 NX EX 10
APPEND	Añadir texto al final de un String	APPEND log " nueva entrada"
MULTI / EXEC	Bloque de transacción atómica	MULTI → comandos → EXEC
WATCH	Optimistic locking (cancelar si cambia)	WATCH counter:users
DECRBY	Decrementar contador en N	DECRBY counter:users 5

Parte B PK Distribuida



Objetivo

Implementar un **generador de claves primarias distribuidas** mediante Redis, analizar la **atomicidad** de las operaciones de incremento y diseñar **estrategias de resiliencia** ante caídas del servicio.

Al finalizar esta parte serás capaz de:

- Diseñar un esquema de claves Redis para generar IDs únicos globales.
- Implementar un generador secuencial de PKs con atomicidad garantizada.
- Analizar las consecuencias de la concurrencia en la generación de IDs.
- Proponer estrategias de fallback ante caídas temporales de Redis.
- Diseñar mecanismos de recuperación de datos en caso de pérdida total.

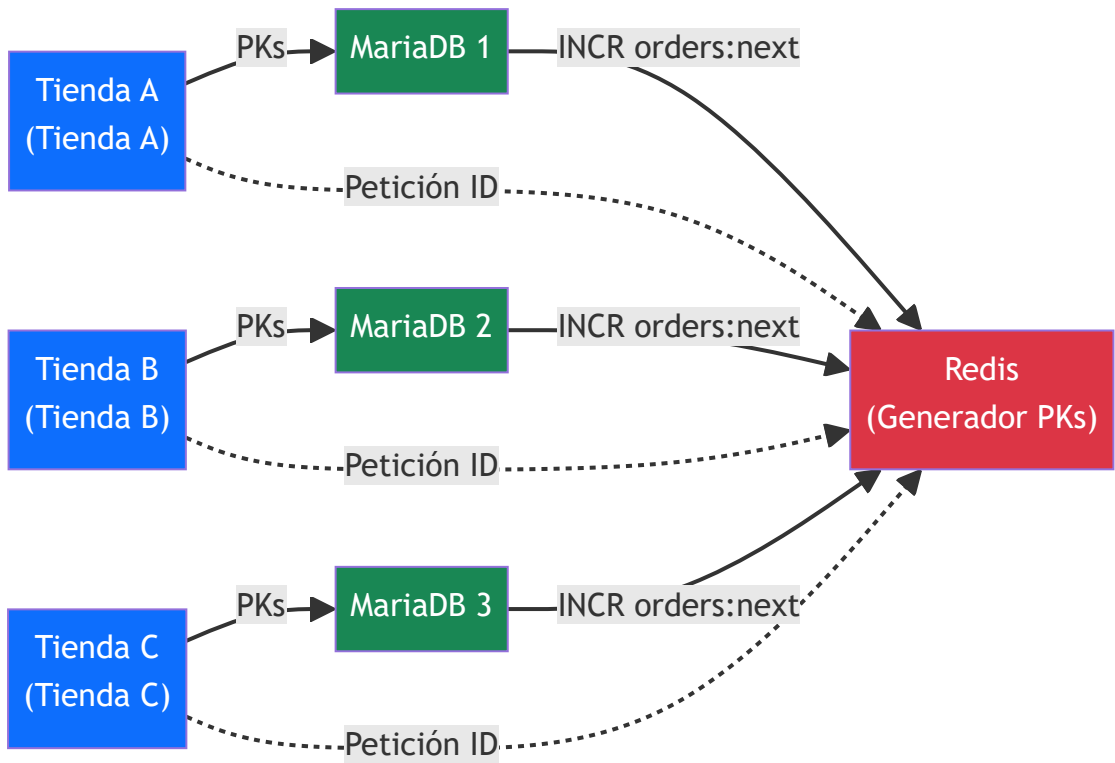
Enunciado

Caso práctico:

Trabajas en un **tienda online con múltiples vendedores** con 5 tiendas online. Cada tienda tiene su propia base de datos MariaDB, pero necesitas un sistema de **IDs único global** para pedidos compartidos entre tiendas. Diseña e implementa un **generador de PKs distribuidas con Redis**.

Diagrama del escenario

El siguiente diagrama muestra la arquitectura con múltiples vendedores y cómo Redis actúa como generador centralizado de IDs únicos para todas las tiendas:



Explicación: Cada vendedor gestiona sus datos en su propia instancia de MariaDB, pero delega la generación de IDs únicos a Redis, que garantiza unicidad global mediante operaciones atómicas.

Requisitos previos

Componente	Versión mínima	Comprobación
Docker Desktop	4.x	<code>docker --version</code>
JetBrains DataGrip	2024.x	Abrir la aplicación
.NET SDK	10.0	<code>dotnet --version</code>
Python	3.12+	<code>python --version</code>
Parte A completada	—	Contador distribuido funcional
Contenidos Sesiones 3 y 4	—	Lectura previa realizada

Nota: Debes haber completado la Parte A (Contador distribuido) antes de comenzar esta parte. Los conceptos de INCR atómico son fundamentales para el desarrollo de esta tarea.

Tarea 1: Diseñar el esquema de claves

Tarea de investigación:

Diseña el **patrón de claves Redis** para generar IDs únicos globales por tipo de entidad (usuarios, pedidos, productos). Indica qué **estructura de datos** usarías y por qué.

Preguntas para reflexionar:

- ¿Qué convención de nombrado seguirías para las claves?
- ¿Usarías una sola clave global o claves separadas por entidad?
- ¿Qué estructura de datos Redis elegirías y por qué?
- ¿Cómo afectaría el diseño a la escalabilidad futura?

Pista: Consulta la **Sesión 2, T3.2.4** sobre convenciones de nombrado en Redis para orientar tu diseño.

Tarea 2: Implementar el generador

Tarea de implementación:

Implementa una **función** que dado un nombre de entidad (ej: 'orders') devuelva el **siguiente ID secuencial único**. Garantiza que dos clientes concurrentes **nunca obtengan el mismo ID**.

Especificación de la función:

Interfaz esperada

```
# Entrada: nombre de la entidad (string)
# Salida: ID secuencial único (entero)
#
# Ejemplo:
# generate_id('orders') → 1
# generate_id('orders') → 2
# generate_id('users') → 1 (contador independiente)
# generate_id('orders') → 3
```

Requisitos funcionales:

- Cada entidad debe tener su propio contador independiente.
- La operación debe ser atómica (sin carreras de condición).
- Los IDs deben ser secuenciales (1, 2, 3, ...).

Pista: Consulta la **Parte A, Paso 1 y 2** para revisar la implementación de contadores atómicos con Redis.

Tarea 3: Analizar concurrencia

Escenario de análisis:

Si dos usuarios hacen clic simultáneamente en "**Comprar**" para el mismo producto, y ambos generan un pedido, **¿qué pasaría si el generador de IDs no fuera atómico?** Razona las consecuencias.

Preguntas para reflexionar:

- ¿Qué problema surgiría si se usara GET + SET en lugar de INCR?
- ¿Cómo afectaría a la integridad referencial en MariaDB?
- ¿Qué tipo de errores se producirían en la aplicación?
- ¿Cómo se manifestaría este problema en un entorno de producción con alto tráfico?

Pista: Consulta la **Sesión 2, T3.2.5** sobre la atomicidad de INCR y por qué es fundamental para operaciones concurrentes.

Tarea 4: Estrategia de fallback

Escenario de resiliencia:

Redis cae durante **5 segundos**. En ese intervalo, **3 usuarios intentan hacer pedidos**. Diseña una **estrategia de fallback** para que la aplicación siga funcionando. **¿Qué propiedades garantiza cada estrategia?**

Estrategias a considerar:

- Generación local de IDs con rangos preasignados.
- Uso de UUID como alternativa temporal.
- Cola de mensajes para reintentos diferidos.
- Sistema de reconciliación posterior.

Para cada estrategia, analiza:

- ¿Garantiza unicidad global?
- ¿Qué latencia introduce?
- ¿Qué complejidad añade al sistema?
- ¿Cómo se reconcilian los datos cuando Redis se recupera?

Pista: Consulta la **Parte A, Paso 6** para revisar los mecanismos de tolerancia a fallos implementados en el contador distribuido.

Tarea 5: Recuperación de datos

Escenario de pérdida total:

Si Redis se pierde completamente (volumen de Docker corrupto), **¿cómo recuperarías el último contador válido?** ¿Qué datos guardarías en MariaDB como respaldo?

Preguntas para reflexionar:

- ¿Qué información mínima debes persistir en MariaDB para poder reconstruir el contador?
- ¿Con qué frecuencia deberías sincronizar el contador con MariaDB?
- ¿Qué estrategia usarías para evitar la pérdida de IDs durante la reconstrucción?
- ¿Cómo manejarías la situación si el contador reconstruido es menor que el último ID asignado?

Pista: Consulta la **Sesión 4, T3.4.1** sobre persistencia en Redis para diseñar un mecanismo de backup adecuado.

Preguntas de reflexión

Antes de entregar la práctica, reflexiona sobre las siguientes preguntas para consolidar los conceptos aprendidos:

1. ¿Qué propiedad de INCR garantiza unicidad?

Analiza por qué INCR es seguro en entornos concurrentes y qué garantía técnica ofrece Redis.

2. ¿Qué pasaría si usaras GET + SET en lugar de INCR?

Considera el escenario en el que dos clientes leen el mismo valor y luego escriben. ¿Qué resultado obtendrían?

3. ¿Cómo manejarías múltiples vendedores con contadores separados?

Diseña un esquema de claves que permita a cada tienda tener su propio espacio de IDs sin interferir con las demás.

4. ¿Cuándo usarías INCR vs SETNX?

Analiza las diferencias entre incremento atómico y establecimiento condicional. ¿En qué casos es preferible uno u otro?

5. ¿Qué alternativas existen a Redis para IDs distribuidos?

Investiga UUID, Snowflake y ULID. Compara sus ventajas e inconvenientes frente al enfoque de Redis con INCR.

6. ¿Cómo escalarías a múltiples data centers?

Si la aplicación creciera a nivel internacional, ¿cómo garantizarías unicidad de IDs entre centros de datos separados geográficamente?

7. ¿Qué pasaría si el contador se queda sin espacio?

Considera el límite de 64 bits de INCR. ¿Qué estrategia implementarías para evitar este problema en producción?

8. ¿Cómo sincronizarías contadores entre Redis primario y réplicas?

En una configuración con replicación, ¿cómo afectaría la latencia de replicación a la unicidad de IDs? ¿Qué estrategias existen para mitigar este riesgo?



Arquitectura de Datos

4.º Curso, Grado en Ingeniería Informática

Curso académico 2026/2027

Human Language and Accessibility Technologies Group (HULAT).

Departamento de Informática

Universidad Carlos III de Madrid